SMU SINGAPORE MANAGEMENT UNIVERSITY | School of Computing and Information Systems

Computational Thinking with Python

Jean CHEN **James KOH**

{jeanchen, jameskohby}@smu.edu.sg

Precision & Limits

(optional)

Strange Thing


```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>>print('{:.30f}'.format(0.1))
0.100000000000000005551115123126
```

Why?

Optional

- Exercise: *Strange Thing*

© MITB, SMU

Precision

Base 10

$$\frac{1}{3} = 0.3333\ldots$$

$$\frac{2}{3} = 0.6666\ldots$$

Optional

Can not be exactly represented.

https://docs.python.org/3/tutorial/floatingpoint.html

© MITB, SMU

Precision

Base 10

Recap - for very large or very small numbers:

$$Significant \times Base^{Exponent}$$

Optional

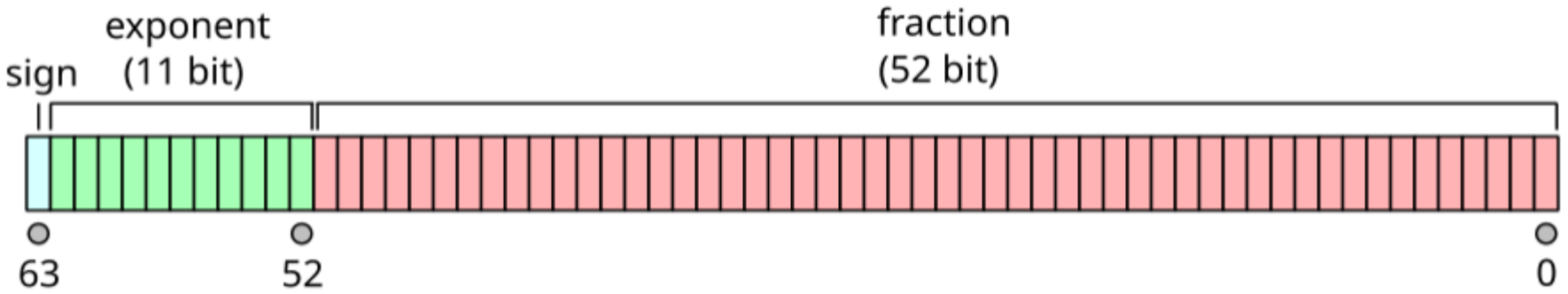
$$12000000 = 1.2 \times 10^{7}$$

$$0.00000012 = 1.2 \times 10^{-7}$$

https://docs.python.org/3/tutorial/floatingpoint.html

© MITB, SMU

Precision



Optional

https://en.wikipedia.org/wiki/Double-precision_floating-point_format

© MITB, SMU

Precision

How to represent $(1472.5)_{10}$ in base 10? - Recap

Base 10

$(1472.5)_{10}$ $2 \times 10^0 = 2$

$7 \times 10^1 = 70$

$4 \times 10^2 = 400$

$1 \times 10^3 = 1000$

$5 \times 10^{-1} = 0.5$

Optional

$\times 10^n$

© MITB, SMU

Precision

How to represent $(17)_{10}$ in base 2? - Recap

Base 2

$(10001)_2$

$1 \times 2^0 = 1$

$0 \times 2^1 = 0$

$0 \times 2^2 = 0$

$0 \times 2^3 = 0$

$1 \times 2^4 = 16$

$\times 2^n$

Optional

© MITB, SMU

Precision

How to represent $(17)_{10}$ in base 2?

Base 2

$(10001)_2$

$$\left\lfloor \frac{17}{2} \right\rfloor = 8$$

$\div 2^n$

$$17 \,\%\, 2 = 1$$
$$8 \,\%\, 2 = 0$$
$$4 \,\%\, 2 = 0$$
$$2 \,\%\, 2 = 0$$
$$1 \,\%\, 2 = 1$$

Optional

© MITB, SMU

Precision

How to represent $(0.00011)_2$ in base 10?

Base 2

$(0.00011)_2$ $0 \times 2^{-1} = 0$

$0 \times 2^{-2} = 0$

$0 \times 2^{-3} = 0$

$1 \times 2^{-4} = 0.0625$

$1 \times 2^{-5} = 0.03125$

Optional

$\times 2^{-n}$

$\div 2^{n}$

© MITB, SMU

Precision

How to represent $(0.1)_{10}$ in base 2?

Base 2

$$0.1 \times 2 = \boxed{0} + \boxed{0.2}$$

$$\boxed{0.2} \times 2 = 0 + 0.4$$

$$0.4 \times 2 = 0 + 0.8$$

$$0.8 \times 2 = 1 + 0.6$$

$$0.6 \times 2 = 1 + 0.2$$

$$0.0001100110 \ldots \times 2^{0}$$

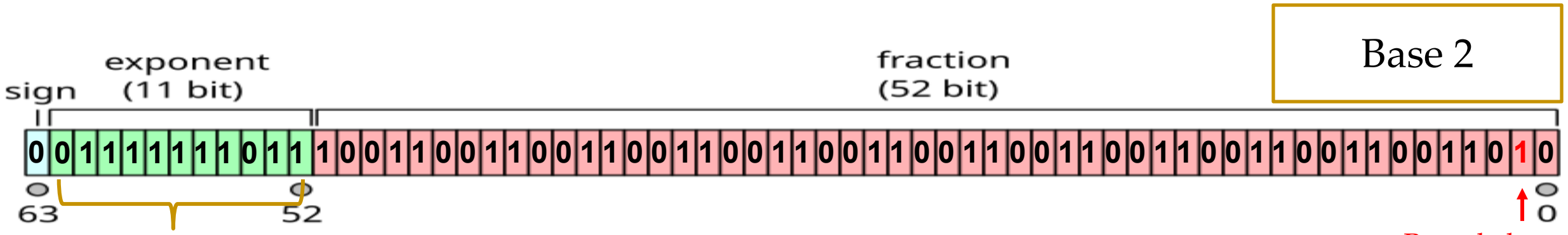
$$1.100110 \ldots \times 2^{-4}$$

Optional

https://docs.python.org/3/tutorial/floatingpoint.html

© MITB, SMU

Precision



Optional

- Normalize $1.100110\ldots \times 2^{-4}$
  - Remove the leading (the leftmost) bit, since it's always 1, and the decimal point, if the case.
  - Adjust its length to 52 bits, only if necessary

https://en.wikipedia.org/wiki/Double-precision_floating-point_format

© MITB, SMU

Precision

Base 2

$(0.1)_{10}$: 0 01111111011 1001100110011001100110011001100110011001100110011010

$(0.2)_{10}$: 0 01111111100 1001100110011001100110011001100110011001100110011010

Optional

+: 0 01111111101 0011001100110011001100110011001100110011001100110100

0	01111111101	0011 0011 0011 0011 0011 0011 0011 0011 0011 0011 0011 0011 0100

In the decimal system, this corresponds to:

0.3000000000000004440892098500626161694526672363281 25

https://numeral-systems.com/ieee-754-converter/

© MITB, SMU

Wooclap!

What is the output of the below code? (Make a guess!)

```
print(0.1 + 0.2 == 0.3)
```

0.30000000000000004

a. True

b. False

Exercise 5-1: Compounded Interest

Assume a bank's annual interest rate is i. If $\$x$ is deposited, the principal and interest after one year will be $\$x(1+i)$.

Assume this bank calculates the interest rate monthly, with a monthly interest rate of $\frac{i}{12}$.

With monthly deposits, the principal and interest after one year will be $\$x(1+\frac{i}{12})^{12}$.

Assume this bank can calculate the interest rate for any given time period. If the deposit period is $\frac{1}{n}$ of a year, the interest rate will be $\frac{i}{n}$ for each period, and the principal and interest after one year will be $\$x(1+\frac{i}{n})^{n}$.

- Does the annual return increase as n increases?

© MITB, SMU

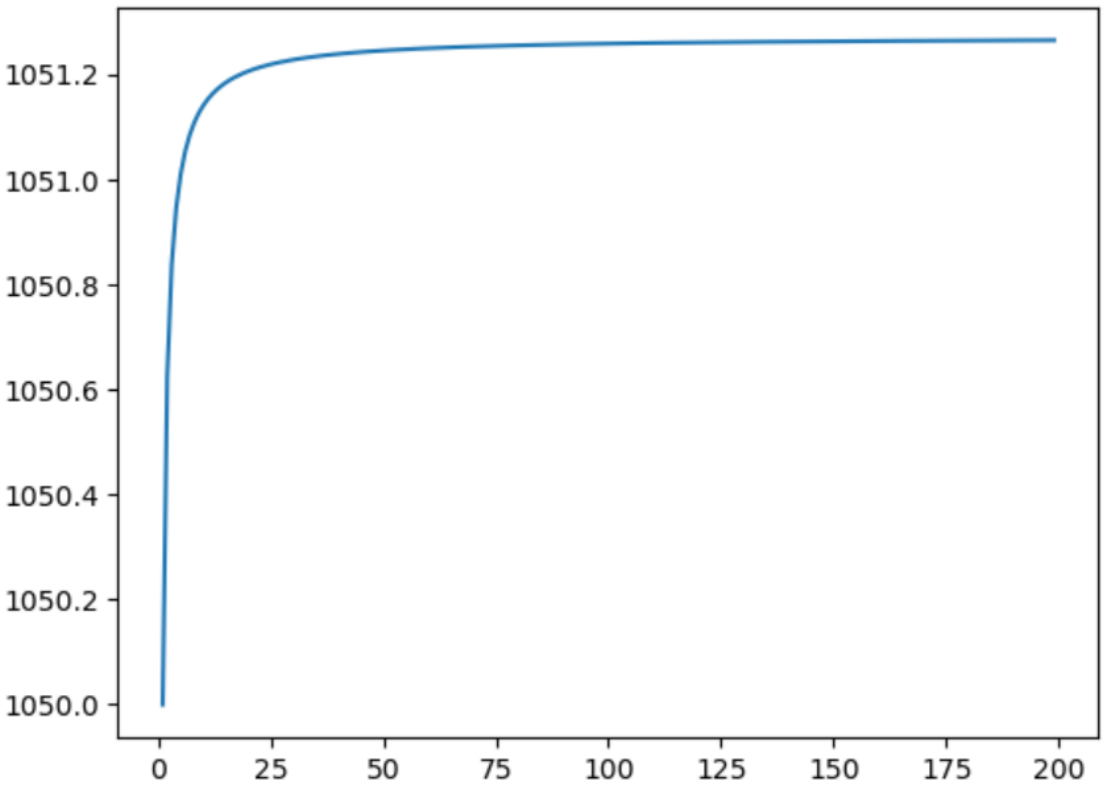
Exercise 5-1: Compounded Interest

Principle: $1000
Annual interest rate: $0.05
After one year ...

Annually	Biannually	Quarterly	Monthly	Daily
$1050	$1050.625	$1050.838	$1050.945	What do you expect?

© MITB, SMU

Natural Log e




```
x = 1000
i = 0.05
n = [nn for nn in range(1,200)]
total = [x * (1 + i/nn) ** nn for nn in n]
plt.plot(n, total)
plt.show()
```


© MITB, SMU

Natural Log e


```
x = 1
i = 1
n = [nn for nn in range(1,200)]
total = [x * (1 + i/nn) ** nn for nn in n]
plt.plot(n, total)
plt.show()
```

$$e = \lim_{n \to \infty} (1 + \frac{1}{n})^n = 2.7 \ldots$$

$$Total = Principle \times e^{interest * time}$$

© MITB, SMU

Limits in Math

$$0.999\ldots = 1$$

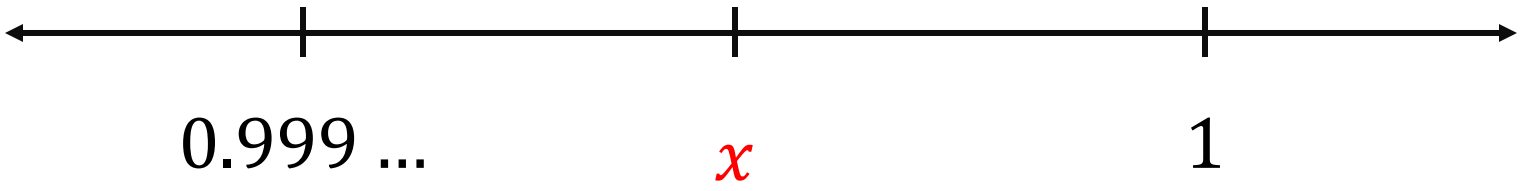


$$\frac{9}{10} + \frac{9}{100} + \frac{9}{1000} + \cdots$$

© MITB, SMU

Limits in Math

$$0.999\ldots \leq 1$$



© MITB, SMU

Limits in Math

$$(0.9)_n \leq 0.999 \ldots \leq x \leq 1$$

$$(0.9)_1 = 0.9$$

$$(0.9)_2 = 0.99$$

© MITB, SMU

Limits in Math

$$(0.9)_n \leq 0.999 \ldots \leq x \leq 1$$

$$(0.9)_n + \frac{1}{10^n} = 1$$

$$(0.9)_3 + \frac{1}{10^3} = 1$$

© MITB, SMU

Limits in Math

$$(0.9)_n \leq 0.999 \ldots \leq x \leq 1$$

$$(0.9)_n + \frac{1}{10^n} = 1$$

$$1 - (0.9)_n = \frac{1}{10^n}$$

© MITB, SMU

Introduce ε

$$|1 - (0.9)_n| = \frac{1}{10^n} < \varepsilon$$

$$\lim_{n \to \infty} (0.9)_n = 1$$

$$|x_n - a| < \varepsilon$$

$$\lim_{n \to \infty} x_n = a$$

© MITB, SMU

Wooclap!

What is the output of the below code?

```
n1, n2 = '1', '2'
flag, sum = 0, ''
while len(sum) < 10:
    if flag:
        sum += n1
    else:
        sum += n2
    flag = 1 - flag
print(sum)
```

1. flag starts at 0 ⇒ treat as false
   ⇓ adds n2 = 2, so 1st char = 2
2. flag = 1 - flag = 1 - 0 = 1
   ⇓ flag = true
   ⇓ adds n1 = 1, so next char is = 1
3. flag = 1 - flag = 1 - 1 = 0 ⇒ false
   ⇓ 3rd char = 1
   ⇓ repeats alt. numbers until str was len = 10

a. 212121212

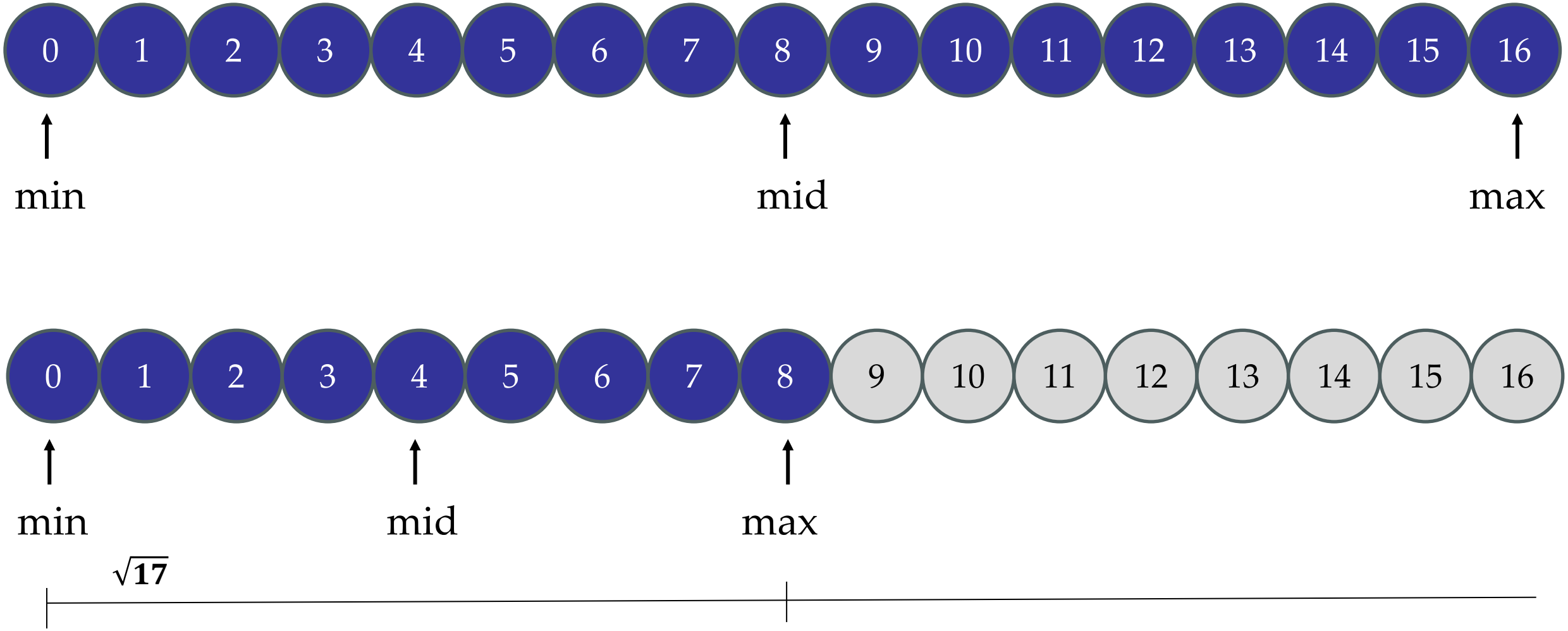
b. 2121212121

c. 222222222

d. 2222222222

Binary Search

One way to find $\sqrt{16}$



© MITB, SMU

Exercise 5-2: Find Square Root

Follow the idea of previous slide and write a python program to use binary search to find square root of 16.

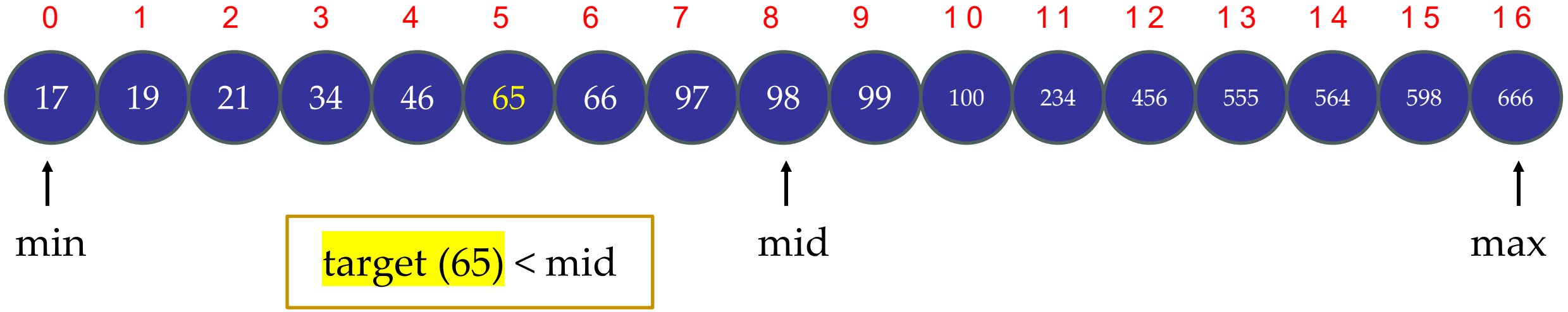
Can we find the exact square root of any number?

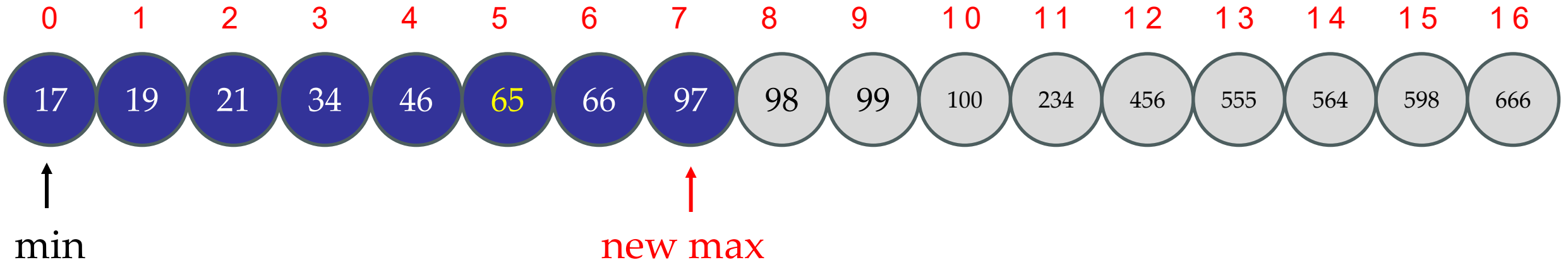
Modify your program to find square root **up to given precision**.

Modify your program to find **3rd root**.

© MITB, SMU

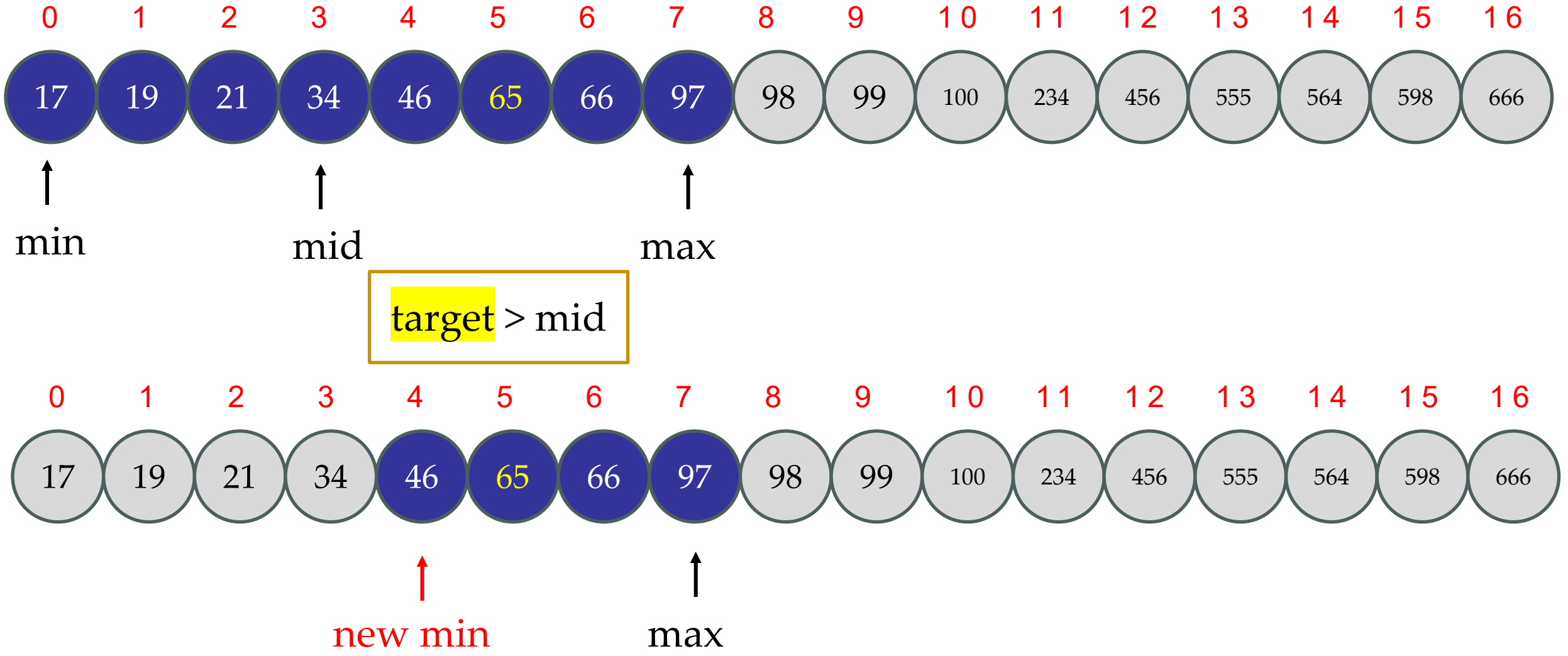
Find 65





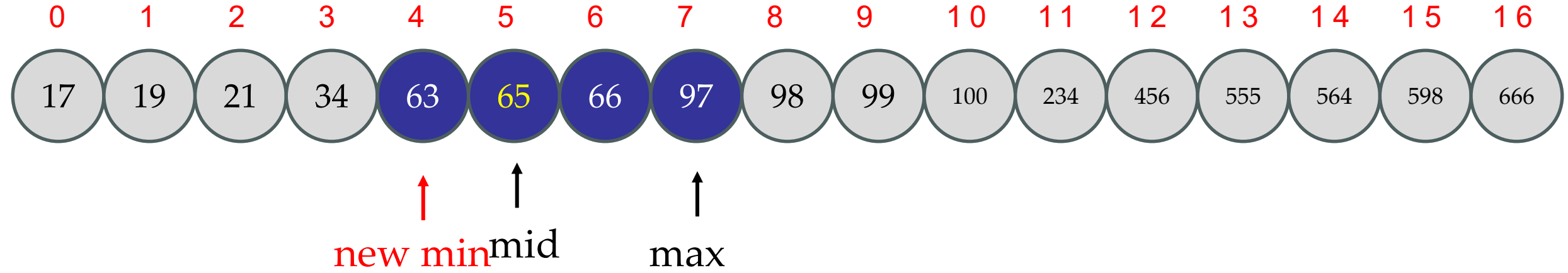
© MITB, SMU

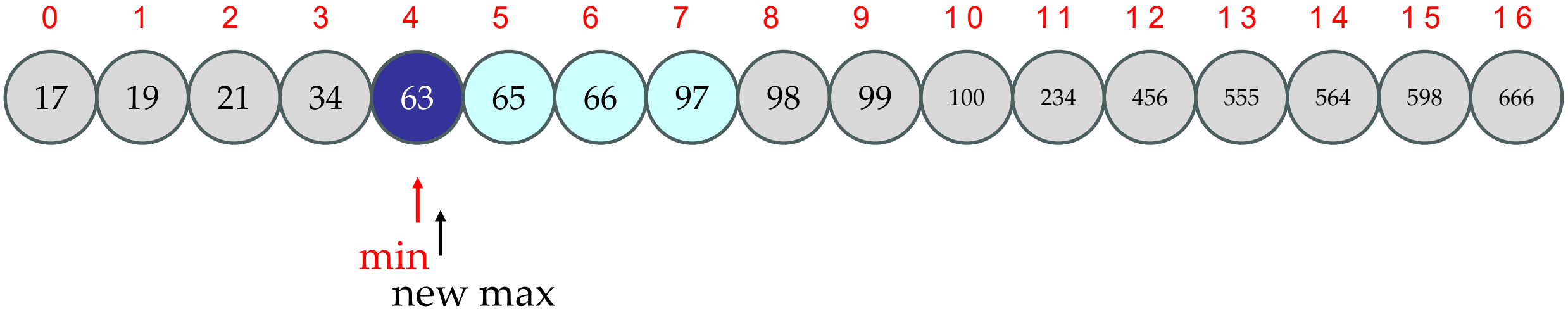
Find 65



© MITB, SMU

What if “Find 62”>





Exercise 5-3: Find the target from a sorted list

Follow the idea of previous slide and write a python program to use binary search to find 65 from the below sorted list.

[17, 19, 21, 34, 46, 65, 66, 97, 98, 99, 100, 234, 456, 555, 564, 598, 666]

© MITB, SMU

Exercise 5-3 (expanded)

- Why not min_idx < max_idx?
- Search from min_idx to max_idx, inclusively
  - [min_idx, max_idx]
- <=
  - stop while loop when [idx + 1, idx]
- <
  - stop while loop when [idx, idx]
  - what if the target is at the idx?

```
def BinarySearch(lst, target):
    min_idx, max_idx = 0, len(lst) - 1
    while min_idx <= max_idx:
        mid_idx = (min_idx + max_idx) // 2
        if lst[mid_idx] == target:
            return mid_idx
        elif lst[mid_idx] < target:
            min_idx = mid_idx + 1
        else:
            max_idx = mid_idx - 1
```


© MITB, SMU

Exercise 5-3 (expanded)

- Why not min_idx = mid_idx?

- Search from min_idx to max_idx, inclusively
  - [min_idx, max_idx]
- Already compared the value at mid_idx with target
- Needs extra steps

```
def BinarySearch(lst, target):
    min_idx, max_idx = 0, len(lst) - 1
    while min_idx <= max_idx:
        mid_idx = (min_idx + max_idx) // 2
        if lst[mid_idx] == target:
            return mid_idx
        elif lst[mid_idx] < target:
            min_idx = mid_idx + 1
        else:
            max_idx = mid_idx - 1
```


© MITB, SMU

(scenario – initial)

idx

0	1	2	3	4	5	6	7	8	9
2	3	6	10	11	16	29	30	31	39



min = 0, max = len(lst) – 1 = 9

mid = 4

```
while min_idx <= max_idx:
    mid_idx = (min_idx + max_idx) // 2
    if lst[mid_idx] == target:
        return mid_idx
    elif lst[mid_idx] < target:
        min_idx = mid_idx + 1
    else:   # lst[mid_idx] > target
        max_idx = mid_idx - 1
```


© MITB, SMU

(scenario – different targets)

0	1	2	3	4	5	6	7	8	9
2	3	6	10	11	16	29	30	31	39




```
while min_idx <= max_idx:
    mid_idx = (min_idx + max_idx) // 2
    if lst[mid_idx] == target:
        return mid_idx
    elif lst[mid_idx] < target:
        min_idx = mid_idx + 1
    else:   # lst[mid_idx] > target
        max_idx = mid_idx - 1
```

Suppose our target is 10 or less

0	1	2	3	4	5	6	7	8	9
2	3	6	10	11	16	29	30	31	39



Suppose our target is 16 or more

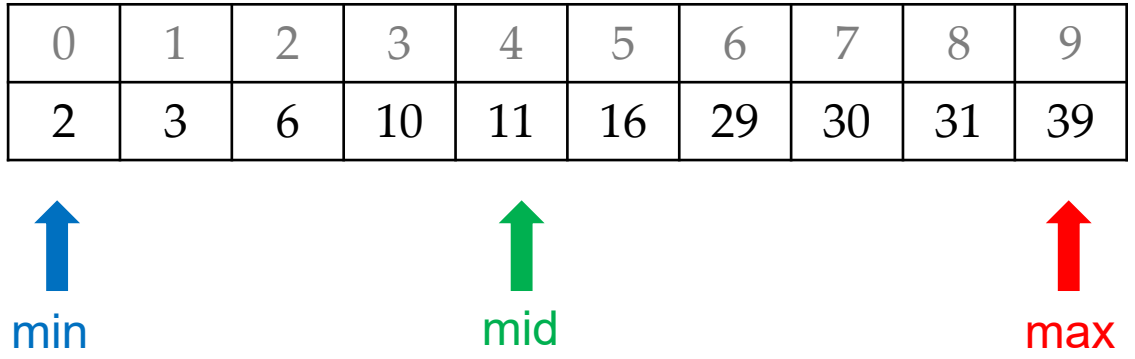
0	1	2	3	4	5	6	7	8	9
2	3	6	10	11	16	29	30	31	39

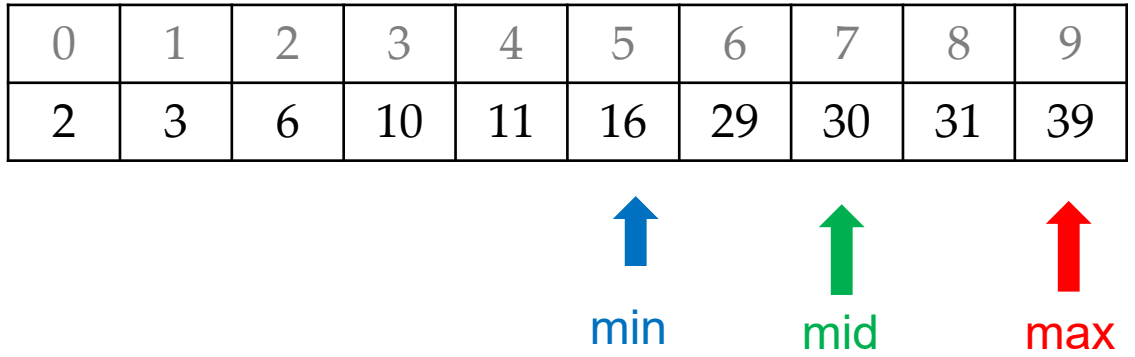


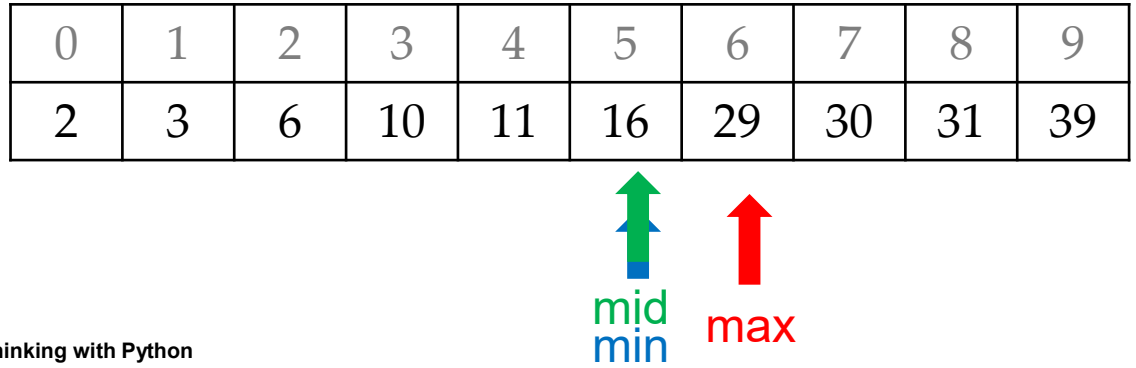
© MITB, SMU

(scenario – target 16)

Suppose our target is 16








```
while min_idx <= max_idx:
    mid_idx = (min_idx + max_idx) // 2
    if lst[mid_idx] == target:
        return mid_idx
    elif lst[mid_idx] < target:
        min_idx = mid_idx + 1
    else:   # lst[mid_idx] > target
        max_idx = mid_idx - 1
```

Carbon emissions due to efficiency

- When n is small, there is little impact from using an efficient vs inefficient solution. Power consumption (and hence carbon emissions) would be similar.
  - Eg. n=17 in our example: 10^{-9} kg CO_2 (linear search) vs 10^{-9} kg CO_2
- When n is large, the efficiency plays a huge difference
  - Eg. n=100000000: 10^{-9} kg CO_2 (linear search) vs 10^{-5} kg CO_2

*Obtained using codecarbon library. Note that the exact computed numbers would differ across machines, and within across different runs on the same machine.

© MITB, SMU

Wooclap!

What is the output of the below code?

```
def ownfilter(list, func):
    def ownfilter(list, func):
        idx = 0
        while idx < len(list):
            if not func(list[idx]):
                list.pop(idx)
            idx += 1
        return list

pr
    print(ownfilter([1, 2, 1, 1, 3, 1, 2, 3, 2, 2, 1, 1, 3], lambda x: x != 1))
```

a. [2, 3, 2, 3, 2, 2, 3]

b. [2, 1, 3, 2, 3, 2, 2, 1, 3]

c. [2, 1, 1, 3, 2, 3, 2, 2, 1, 1, 3]

d. [1, 2, 1, 1, 3, 1, 2, 3, 2, 2, 1, 1, 3]

function

Loops with uneven increment

Wooclap!

What is the output of the below code?

```
sum, count = 21, 0
while sum:
    sum -= count
    count += 1
print(count)
```

	sum	count
1	21	1
2	20	2
3	18	3
4	15	4
5	11	5
6	6	6
7	7	0

a. 0

b. 6

c. 7

d. Code will not terminate

Exercise 5-4: Flash Flooded Skyline (not tested)

Flash flood is a serious problem for modern cities. A team of IT professionals are engaged to describe how severe flash floods can be, and you are assigned to the core task which computes the height of the flash floods at different parts of a city in a simulated setting with a specified rainfall. The following assumptions help you simply the problem.

- Buildings are lined up in a single file, and all building are of **20 meters wide**, but their **height can be different**. Buildings are next to one another without any gap.

© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)

- When it rains, rainwater falling on one tall building will **flow to the shorter buildings** if there are. If the building on the left side is shorter but the building on the right side is taller, water will flow to the left side, and vice versa. If both buildings on the left side and on the right side are shorter, **half** of the water flow **to the left side** and the remaining **half** flow **to the right side**.
- There are imaginary walls on both left end and right end, which are taller than all buildings.
- When the space on top of a shorter building is filled with water and it levels up with a taller building beside, water will start accumulate in the space supported by both buildings.

© MITB, SMU

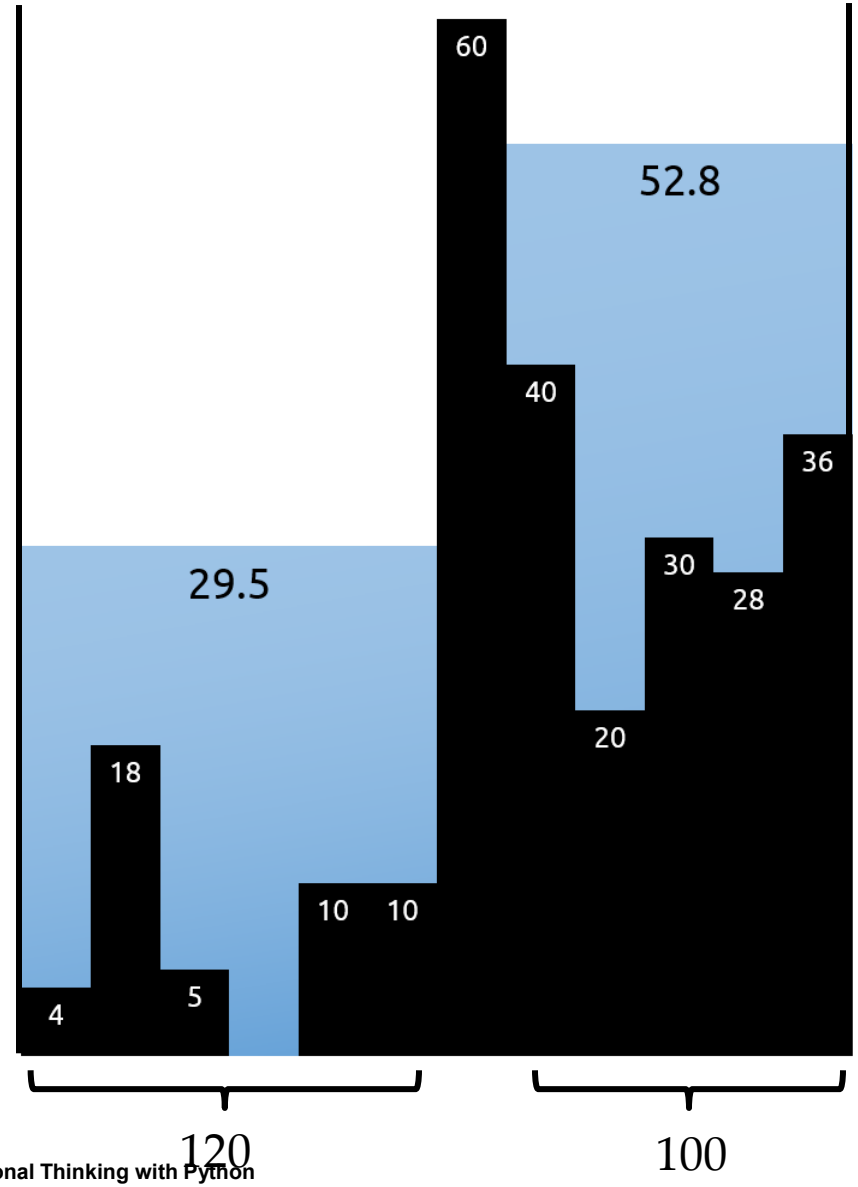
Exercise 5-4: Flash Flooded Skyline (not tested)

- A piece of empty land is considered as a building of height 0, so rainwater from the buildings at the two sides will start accumulating in this space first.
- You are **given the skyline** of the city, i.e., the heights of all buildings, and the list of **possible rainfalls** (in meter, these values look ridiculous as we are doing a simulation from a 3D world into a 2D display).
- You are expected to compute how the skyline of the city looked like with each specified rainfall. A skyline is described by a list of horizontal line segments with height and length.
- height = [4, 18, 5, 0, 10, 10, 60, 40, 20, 30, 28, 36]
- rainfall = [20, 30, 50]

https://medium.com/mitb-for-all/flash-flooded-skyline-3f39000ba8db

© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)



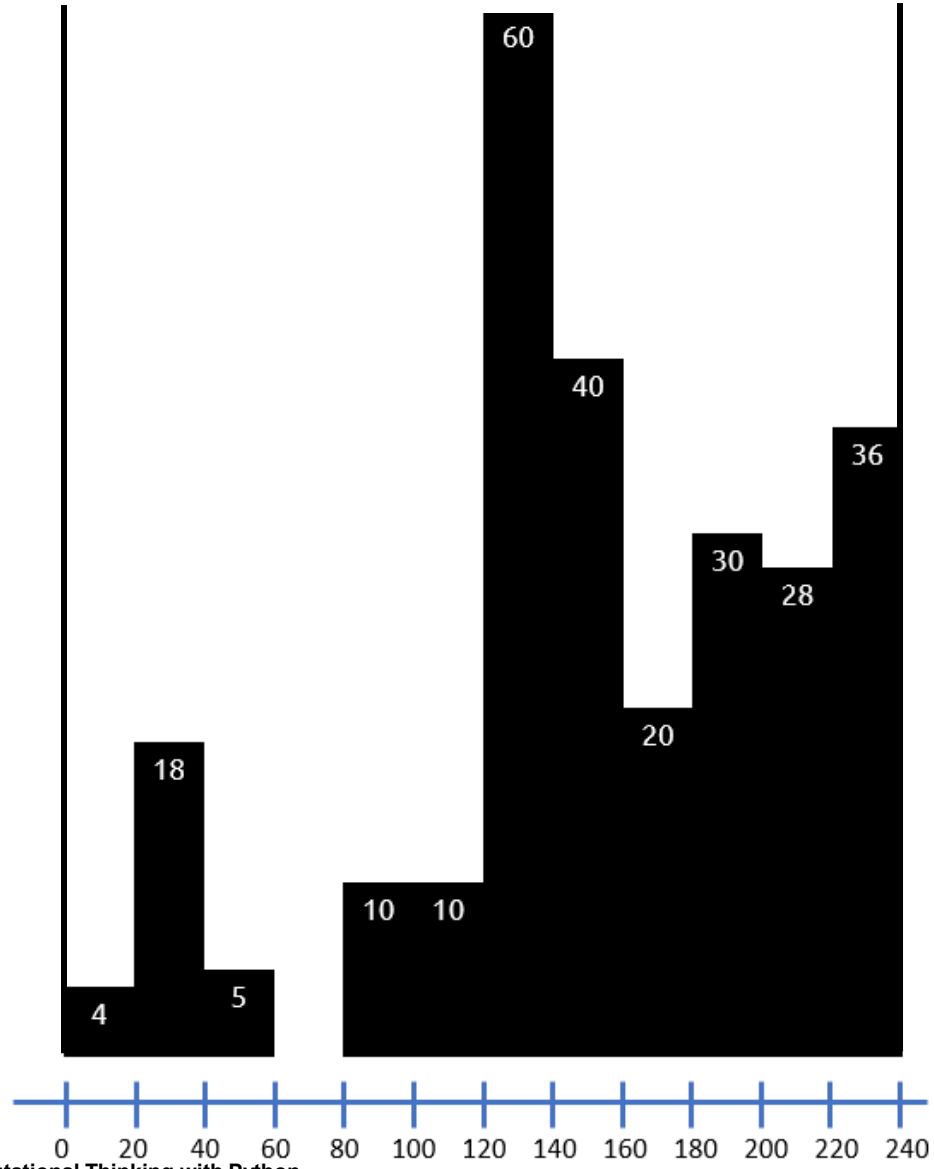
- Output

29.500, 120; 60.000, 20; 52.800, 100

height, width; for each sector

© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)



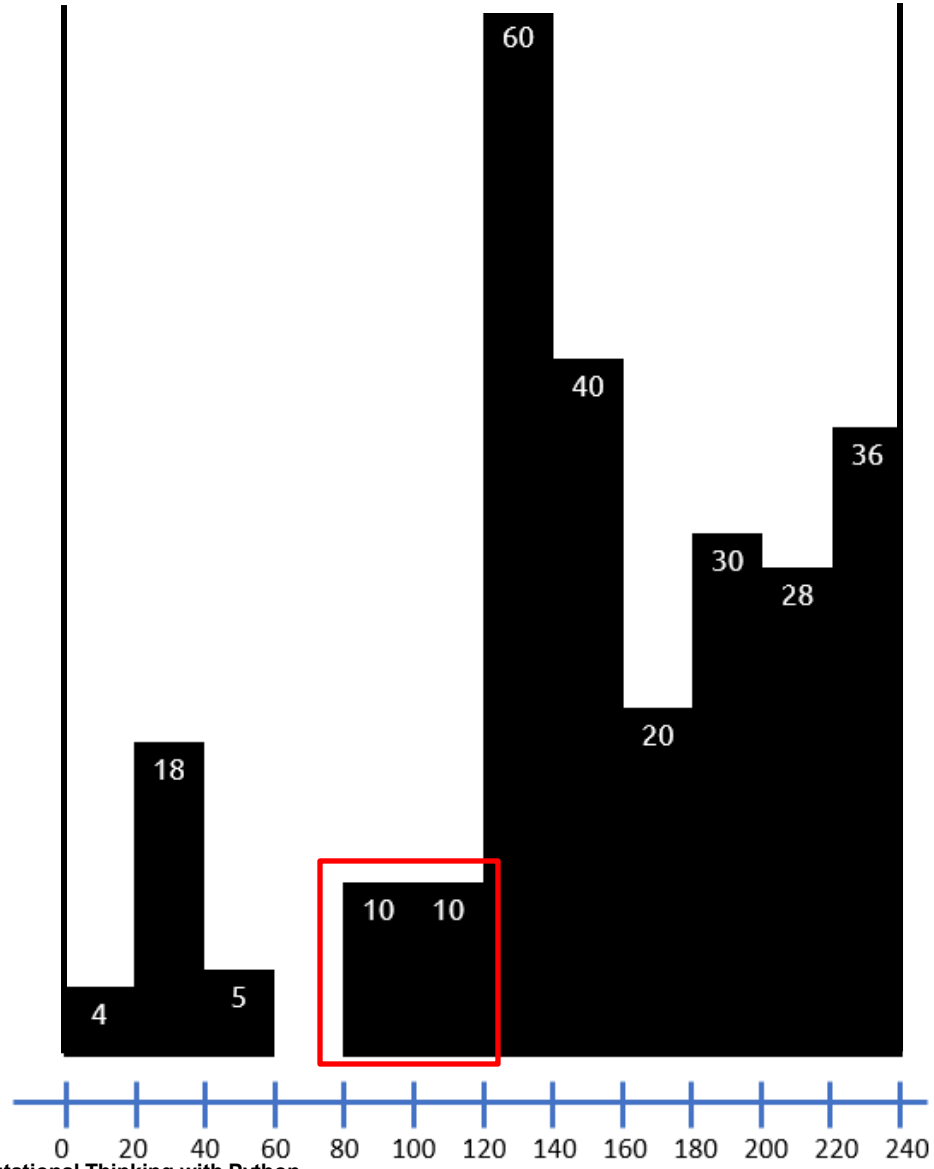
- Step 1: Reformat skyline

[(4.0, 0, 20), (18.0, 20, 40), (5.0, 40, 60), (0.0, 60, 80), (10.0, 80, 100), (10.0, 100, 120), (60.0, 120, 140), (40.0, 140, 160), (20.0, 160, 180), (30.0, 180, 200), (28.0, 200, 220), (36.0, 220, 240)]

(height, start, end) for each sector

© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)



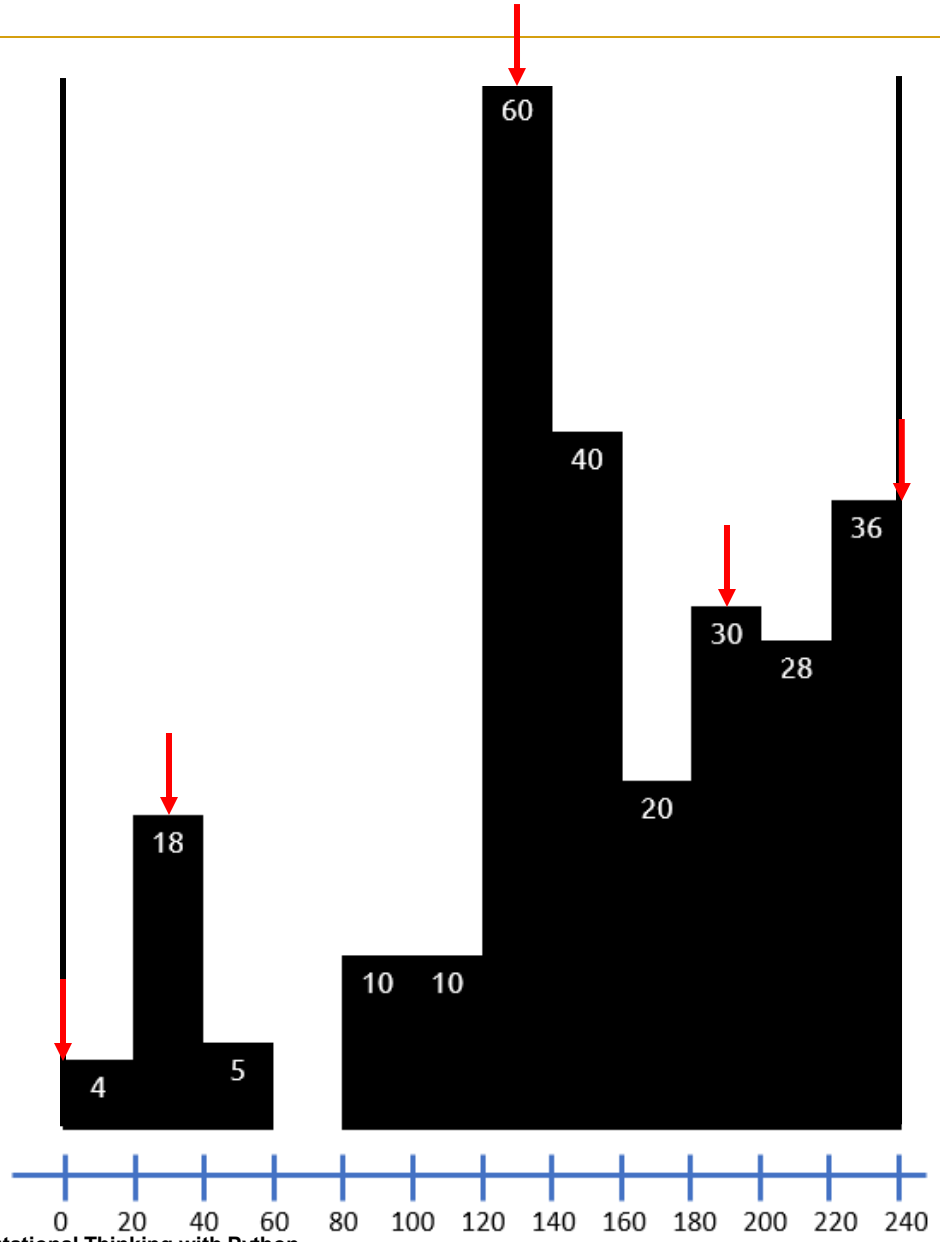
- Step 2: Merge skyline

[(4.0, 0, 20), (18.0, 20, 40), (5.0, 40, 60), (0.0, 60, 80), (10.0, 80, 120), (60.0, 120, 140), (40.0, 140, 160), (20.0, 160, 180), (30.0, 180, 200), (28.0, 200, 220), (36.0, 220, 240)]

(height, start, end) for each sector

© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)



- Step 3: Find peak

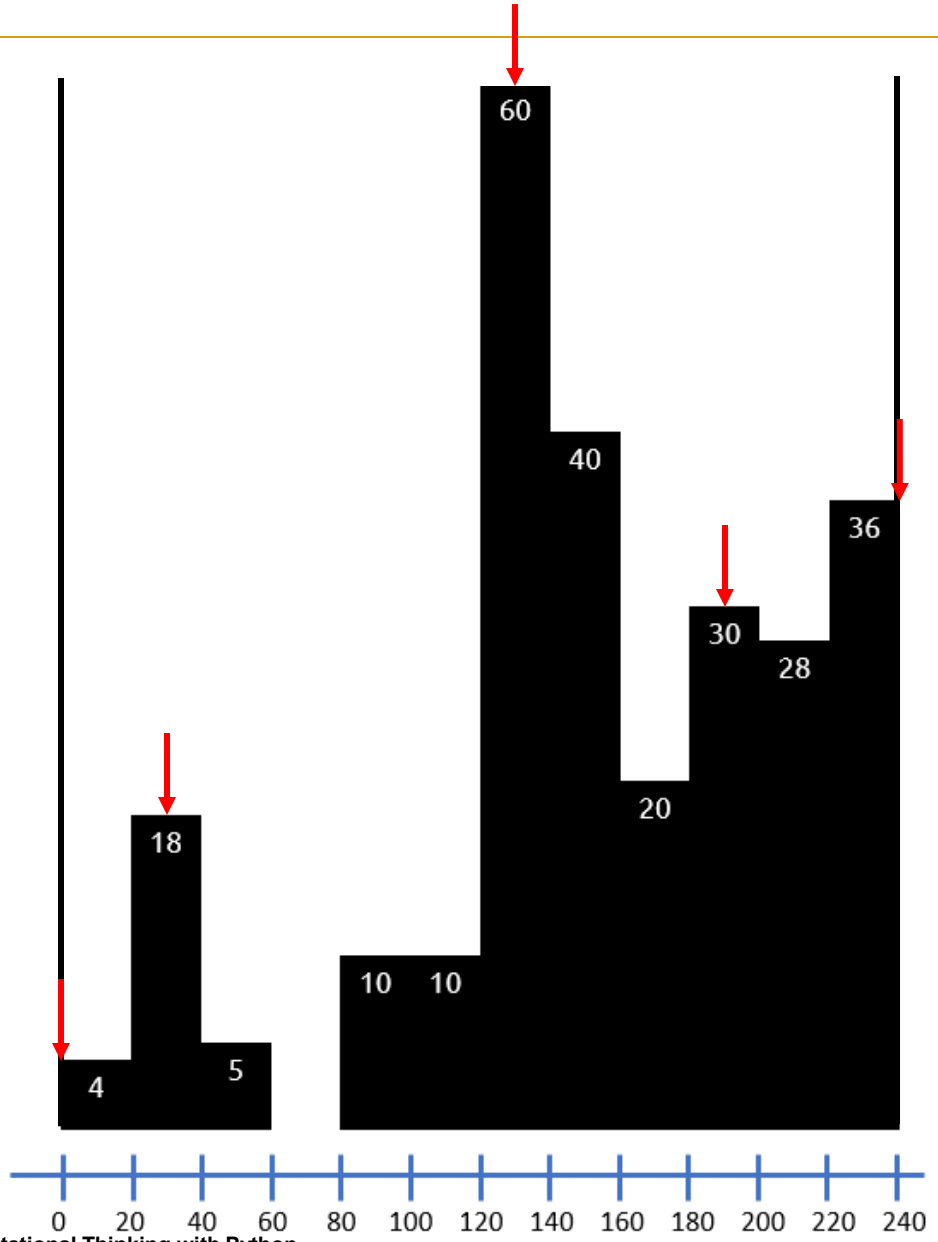
[0, 30, 130, 190, 240]

How to find **peak**?

*Walls on both sides
0, 20 * len(height)

© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)



- Step 3: Find peak

[0, 30, 130, 190, 240]

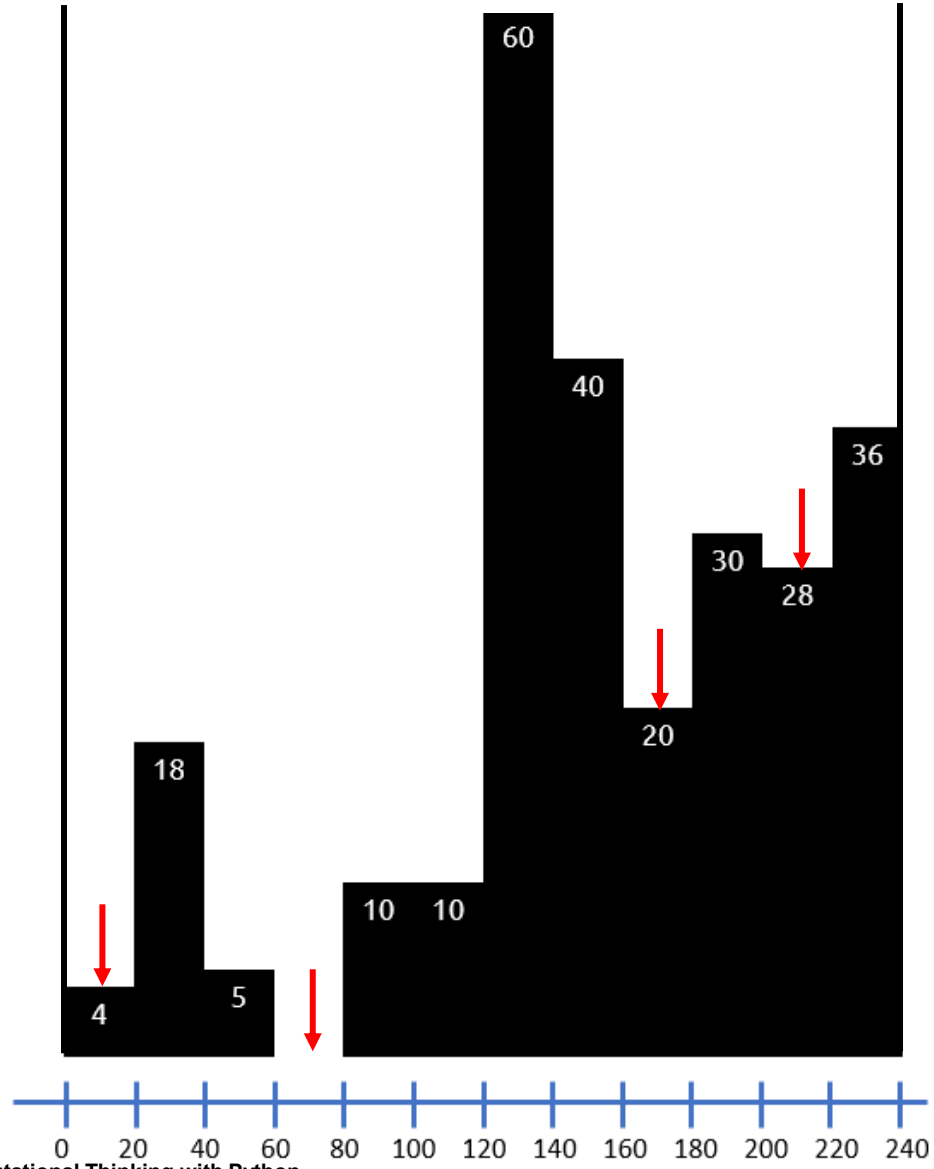
```
skyline[i][0] > skyline[i-1][0]
                and
skyline[i][0] > skyline[i+1][0]
```

⇩

```
(skyline[i][1] + skyline[i][2]) // 2
```


© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)



- Step 4: Find valley

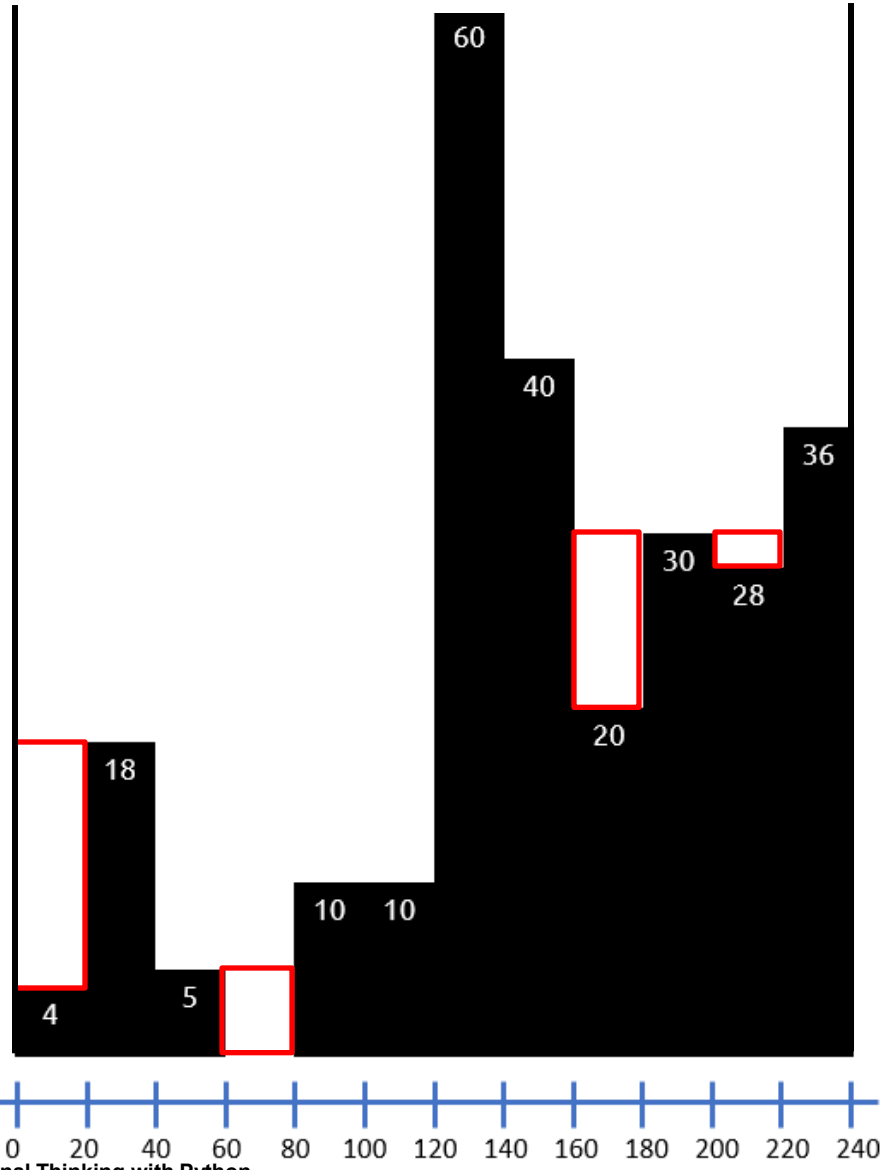
```
skyline[i][0] < skyline[i-1][0]
              and
skyline[i][0] < skyline[i+1][0]
```

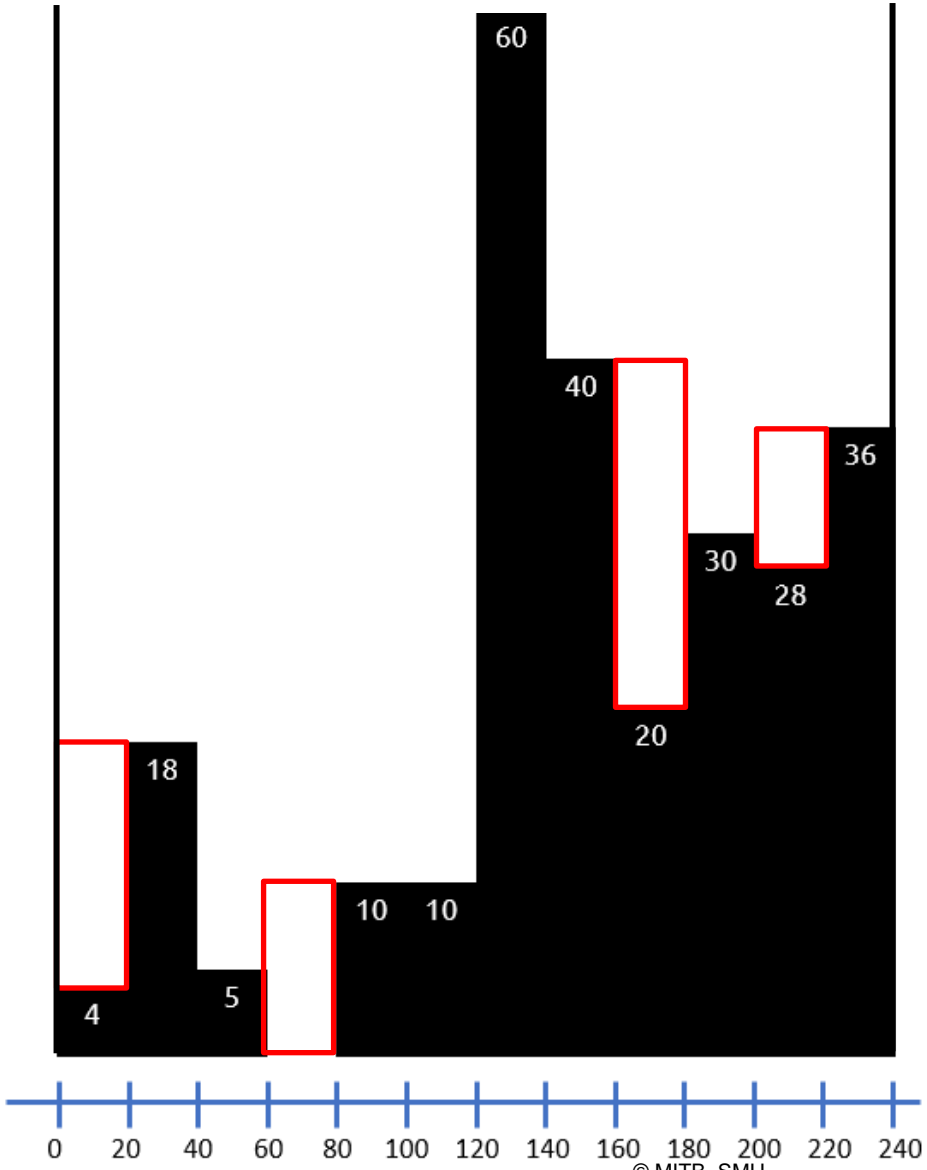
⇩

When to change the skyline?

© MITB, SMU

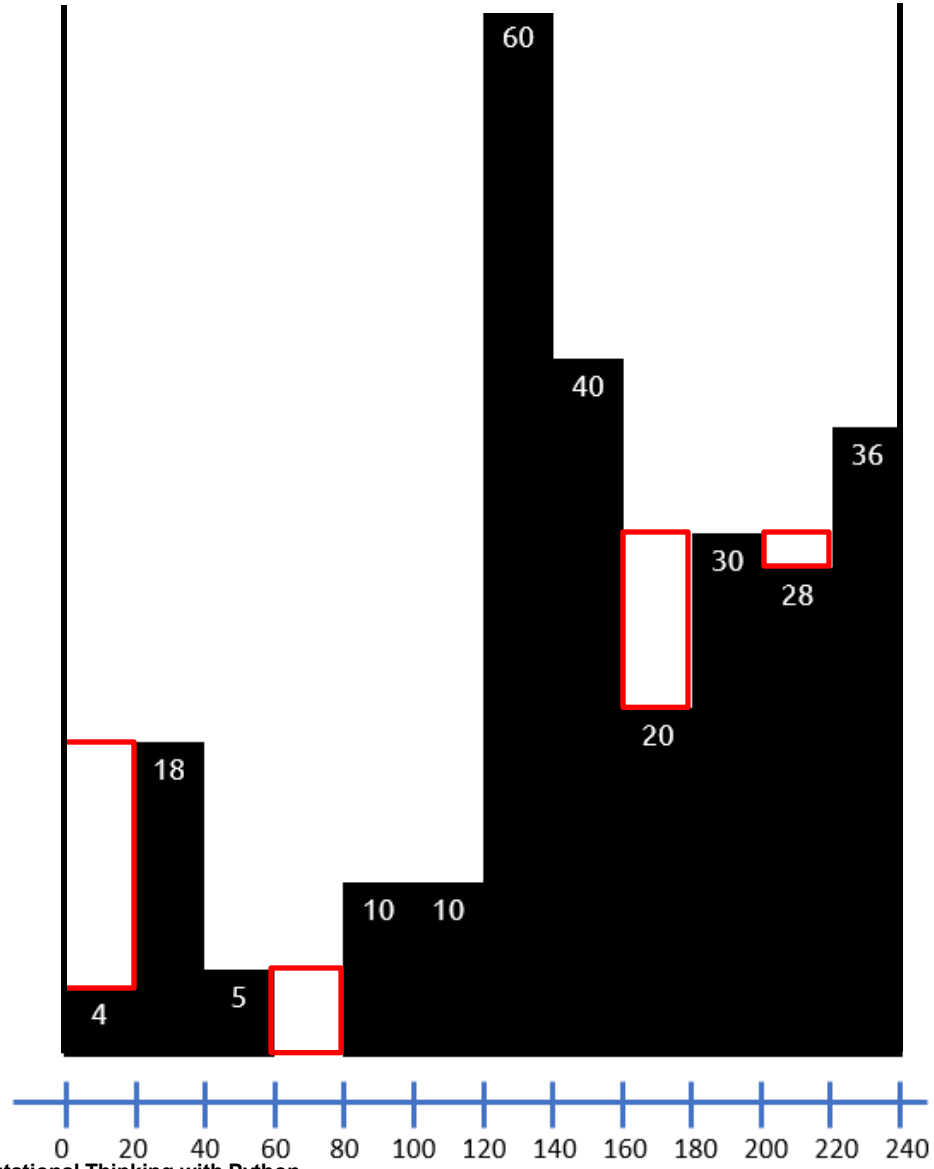
Exercise 5-4: Flash Flooded Skyline (not tested)





© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)



- Step 4: Find valley

[(0, 14.0), (3, 5.0), (7, 10.0), (9, 2.0)]

(section no., height) for each sector

```
skyline[i][0] < skyline[i-1][0]
              and
skyline[i][0] < skyline[i+1][0]
```

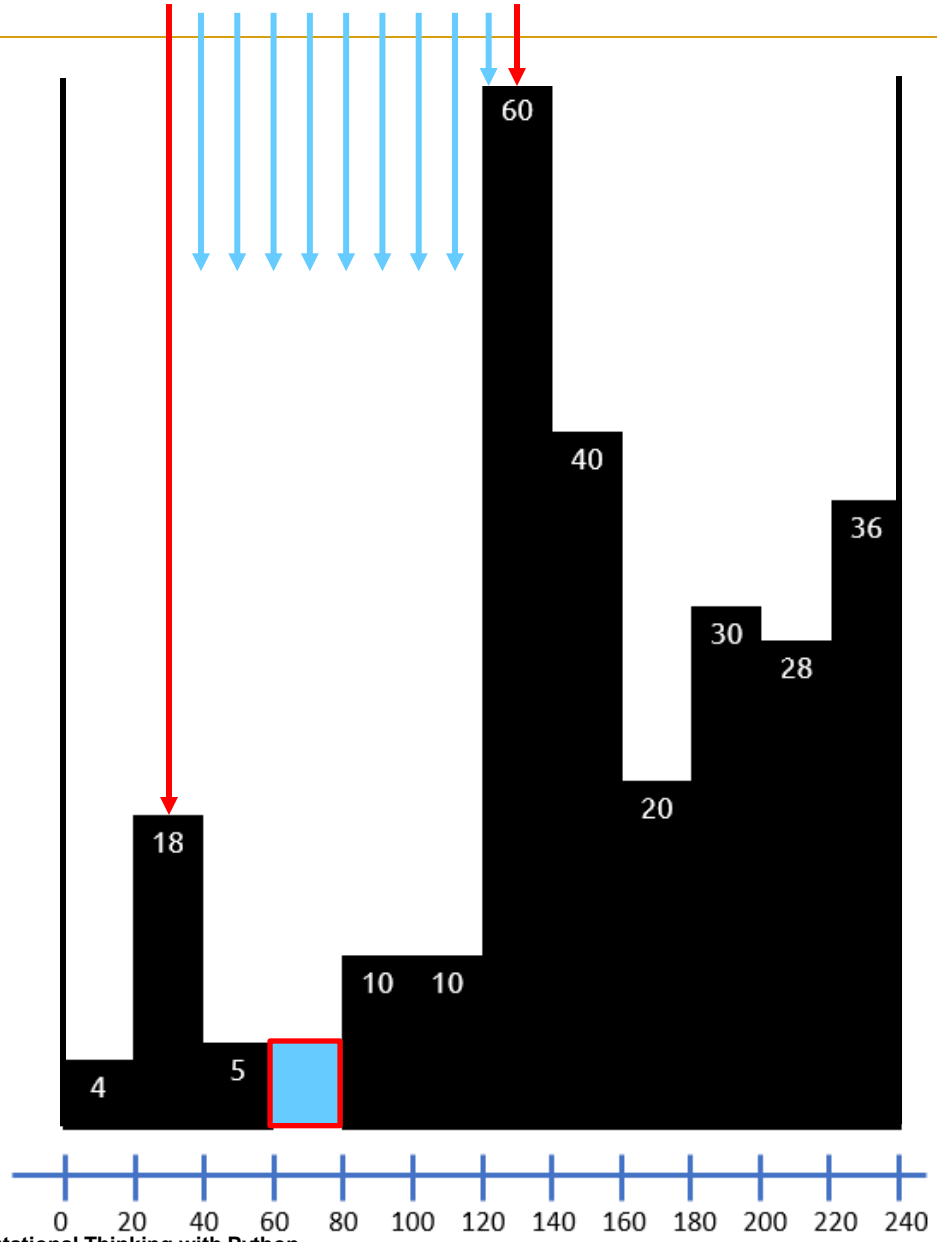
⇩

```
min(diff1,
diff2)
```

How about the valleys next to the wall?

© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)

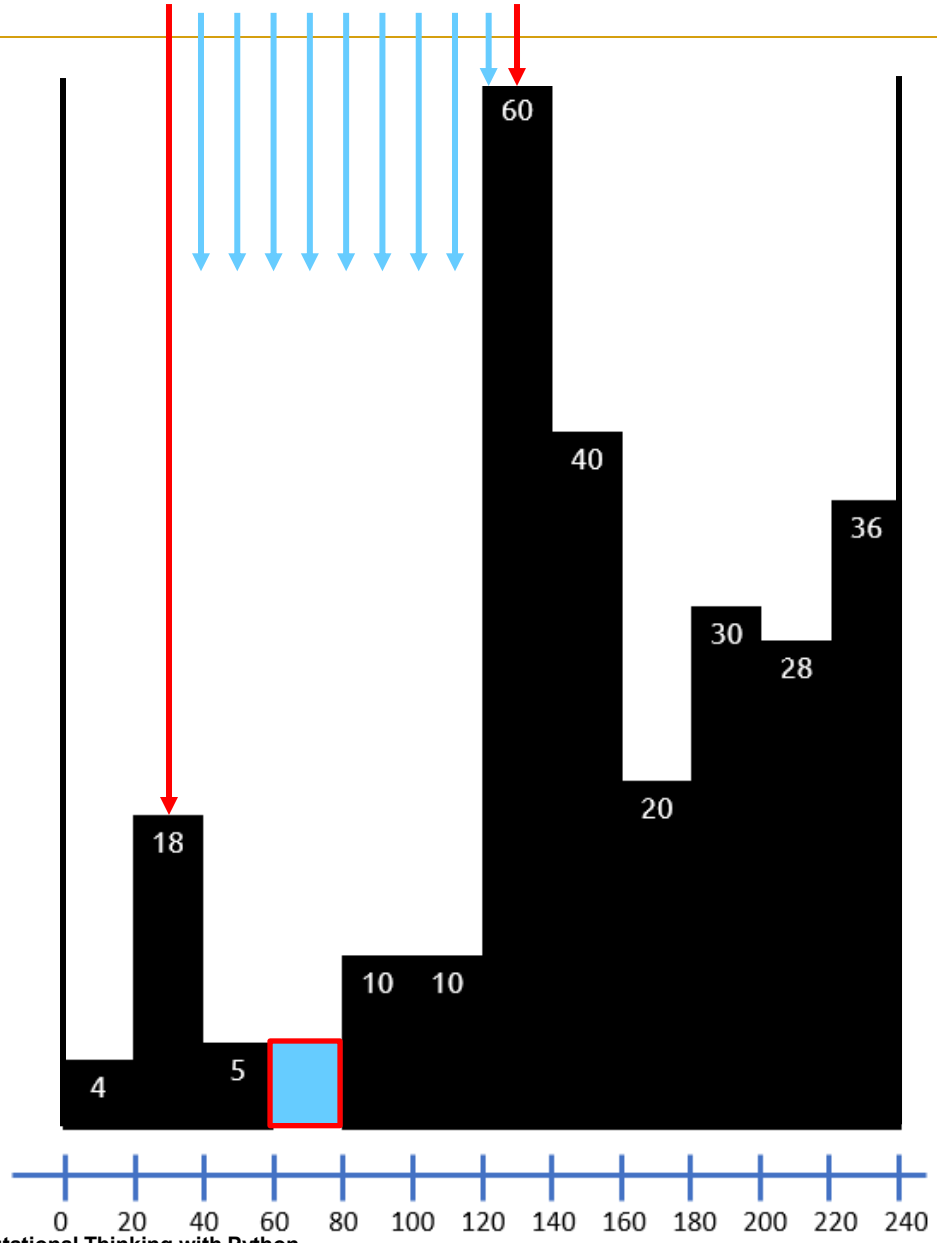


- Step 5: Get the minimum rainfall
  - Get the minimum rainfall that changes the skyline
  - All the rainfall between two peaks next to the valley will fall into that valley

```
(80-60) * 5
    =
(130 - 30) * rainfall
```


© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)

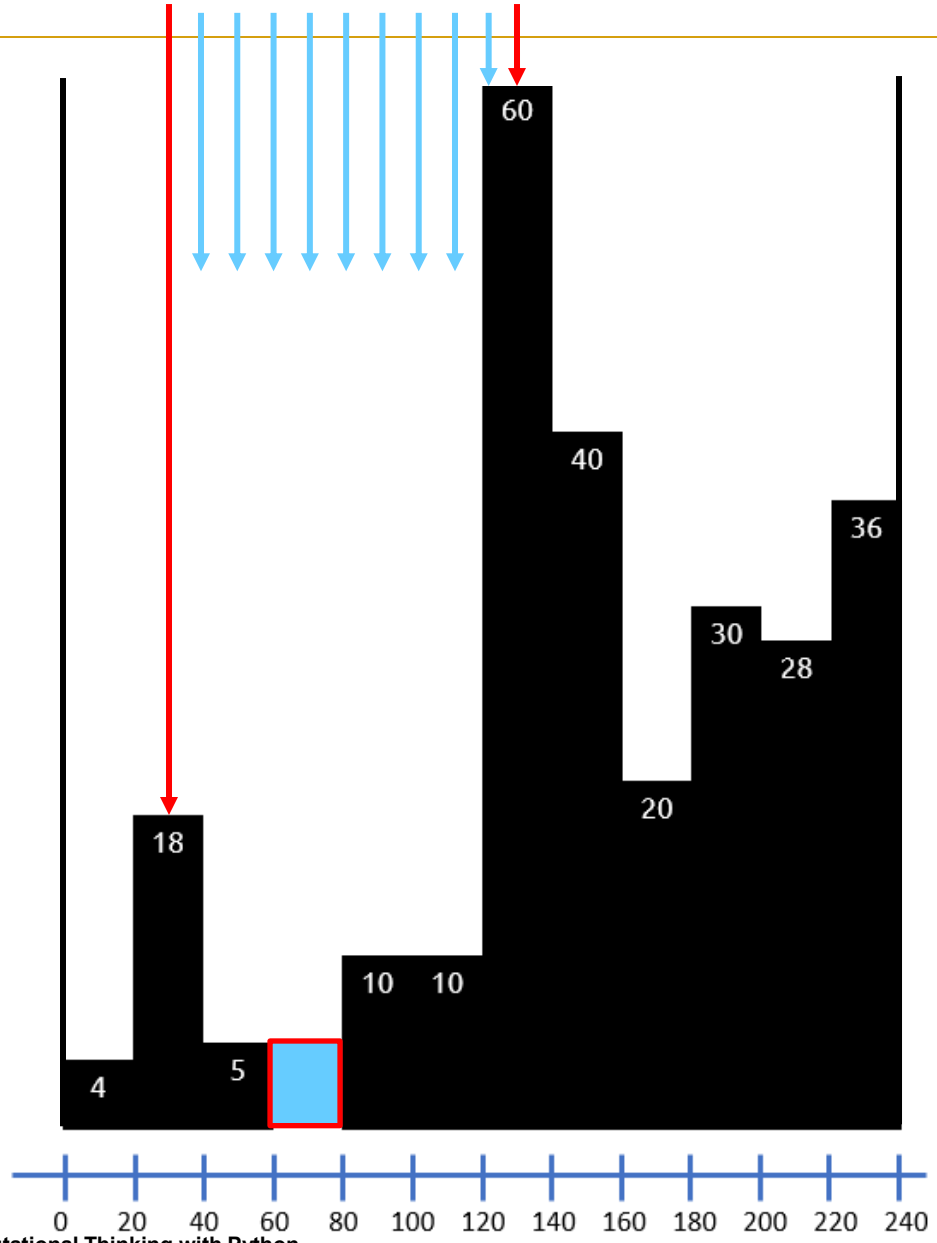


- Step 5: Get the minimum rainfall
  - Get the minimum rainfall that changes the skyline
  - All the rainfall between two peaks next to the valley will fall into that valley

```
(skyline end -start) * valley height
        (80-60) * 5
            =
    (130 - 30) * rainfall
(neighbour peaks position)
```


© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)

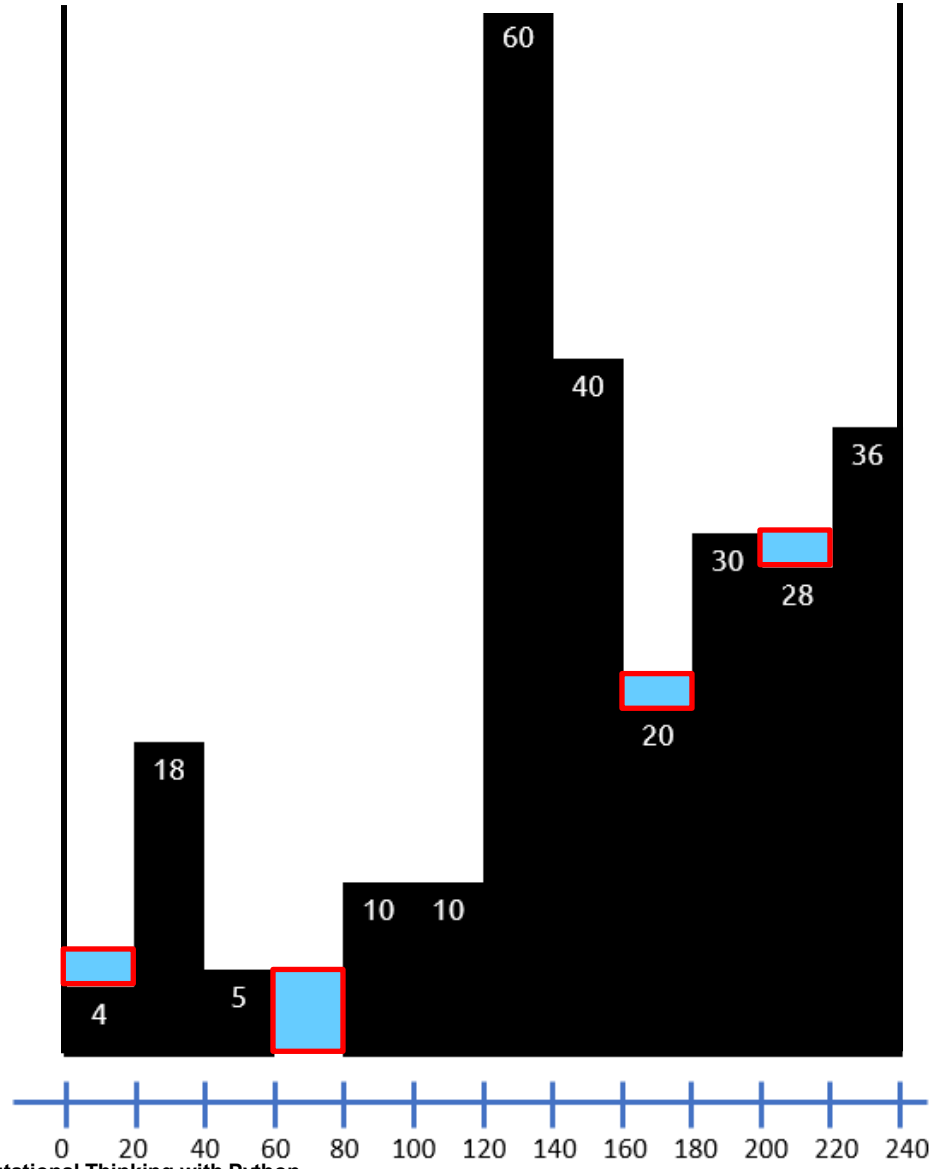


- Step 5: Get the minimum rainfall
  - Get the minimum rainfall that changes the skyline
  - All the rainfall between two peaks next to the valley will fall into that valley

```
min_fall = fall
for i in range(len(valley)):
    j = valley[i][0]
    min_fall = min(min_fall,
(skyline[j][2] - skyline[j][1]) *
valley[i][1] / (peak[i+1] - peak[i]))
```


© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)



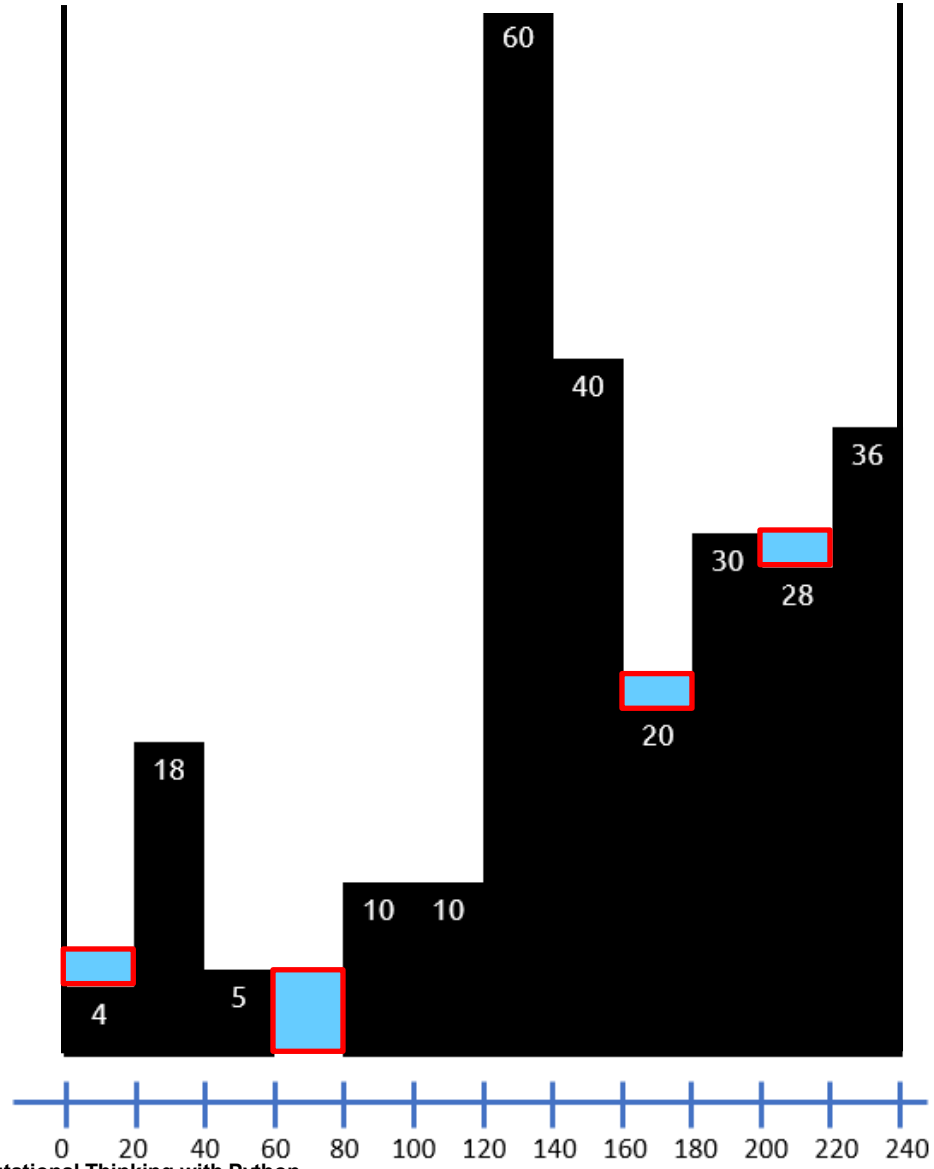
- Step 6: Get the skyline after rainfall

```
for i in range(len(valley)):
    j = valley[i][0]
    skyline[j] = (skyline[j][0] +
(peak[i+1] - peak[i]) * min_fall /
(skyline[j][2] - skyline[j][1]),
skyline[j][1], skyline[j][2])
```

```
New skyline: merge(skyline)
New rainfall: fall - min_fall
```


© MITB, SMU

Exercise 5-4: Flash Flooded Skyline (not tested)



- Repeat step 3 to step 6
- Until fall <= 0

© MITB, SMU

Questions?

Thank You

